

MODULE 27 · DOMAIN 3 - CONTINUOUS IMPROVEMENT FOR EXISTING SOLUTIONS

Improving Deployment Processes

Auditing what exists, and retrofitting safety onto it.

DOMAIN 3 of 4 EXAM WEIGHT 25% MODULE 27 of 44

Start by naming what the current process **cannot do**. Adding tools before that is how you end up with two problems.

Five questions to audit a pipeline

- ◆ **Can it roll back?** If rollback means running the pipeline again, the recovery time is the deployment time.
- ◆ **What stops a bad change?** Tests, approvals, or nothing but the deployer's judgement.
- ◆ **What was deployed, and when?** If nobody can answer, incident timelines are guesswork.
- ◆ **Who can deploy to production?** And is that a permission or a convention?
- ◆ **Does anything watch after the deployment finishes?** Success at deploy time is not success.

ON THE EXAM

The last question catches the most designs. A pipeline that reports green and then stops watching has done half the job.

Retrofitting safety

- ◆ **CodeDeploy** can be added to an existing **Auto Scaling group** without rewriting the application, giving you rollback and hooks.
- ◆ **Rollback triggers on CloudWatch alarms** turn a monitored metric into an automatic reversal, as in module thirteen.
- ◆ **Manual approval actions** add a gate without slowing anything that does not need one.
- ◆ **CodeArtifact** pins dependencies, so a build is reproducible and a compromised upstream package cannot walk in.

ON THE EXAM

Retrofit in the order the gaps hurt. Rollback first, then a gate, then provenance. Each step is independently useful.

EC2 Image Builder

- ◆ A **recipe** is a source image plus components. A **pipeline** builds, tests and distributes on a schedule or a trigger.
- ◆ **Distribution is gated on tests.** An image that fails its tests is never distributed to anyone.
- ◆ **Amazon-managed STIG hardening components** exist, so a compliance baseline is a component rather than a script.
- ◆ It builds **container images to ECR** too, and copies AMIs across Regions and accounts.

ON THE EXAM

The test gate is what makes this a pipeline rather than a build script. Nobody can launch an image that failed its checks.

Making the golden image actually used

- ◆ **Building a golden AMI does not mean anyone launches from it.** Enforcement is a separate problem.
- ◆ **Config rules** `approved-amis-by-tag` and `approved-amis-by-id` detect instances running unapproved images.
- ◆ **An SCP denying `ec2:RunInstances`** unless the AMI carries the approved tag is the preventive version.
- ◆ **Image lifecycle policies** deprecate and delete old images automatically, which is the answer to AMI sprawl.

ON THE EXAM

Detect with Config, prevent with an SCP. The same pairing as modules three and twenty five, applied to images.

Closing the loop on images

- ◆ An Inspector finding of High or Critical can trigger the image pipeline through EventBridge, so a CVE rebuilds the image.
- ◆ The rebuilt image is tested, distributed, and then Auto Scaling instance refresh rolls it out.
- ◆ Recipes and components are **shareable through RAM** and versioned, so one platform team serves the organisation.
- ◆ Patching running instances and rebuilding images are different answers. Long-lived fleets need both.

ON THE EXAM

A vulnerability found by Inspector, rebuilt by Image Builder, rolled out by instance refresh, is the fully closed loop.

Separating deployment from release

- ◆ **Deploying code and enabling a feature can be different events.** Feature flags make the second one a configuration change.
- ◆ **AWS AppConfig**, part of Systems Manager, delivers configuration and flags with validation and gradual rollout.
- ◆ It can roll back automatically on a CloudWatch alarm, exactly like a deployment.
- ◆ The trade is that flags accumulate. Untidied flags become branches nobody dares remove.

ON THE EXAM

A feature that can be switched off without a deployment has a rollback measured in seconds rather than in pipeline runs.

Reading the scenario

THE SCENARIO SAYS

Rollback means running the pipeline again

AMIs are built by hand and drift

Teams launch unapproved images

A new CVE must rebuild every image

Turn a feature off without deploying

Hundreds of old AMIs and snapshots

IT IS POINTING AT

CodeDeploy with alarm rollback

An Image Builder pipeline

Config rule to detect, SCP to prevent

Inspector finding to EventBridge

Feature flags with AppConfig

Image lifecycle policies

ON THE EXAM

Ask what the process cannot do, then fit a mechanism to that gap. Do not propose a rewrite unless the scenario asks for one.

Meridian Freight deploys a fleet application by running a shell script from an engineer's laptop, which copies artefacts to instances and restarts services. Deployments take 25 minutes. Rolling back means checking out the previous commit and running the script again, which takes another 25.

Nobody can say which version is running where.

What should the solutions architect recommend FIRST?

- A Rewrite the application as containers on ECS with blue/green deployments
- B Move the deployment into CodePipeline with CodeDeploy against the existing Auto Scaling group, with alarm-based rollback
- C Document the shell script and add it to version control
- D Schedule deployments during a nightly maintenance window

◆ PAUSE AND DECIDE

What should the solutions architect recommend FIRST?

A Rewrite as containers

Might be right eventually, and it is a rewrite proposed as a first step. It fixes deployment by rebuilding the application.

C Version control the script

A real improvement and it addresses none of the three complaints. Rollback is still 25 minutes.

B CodePipeline and CodeDeploy retrofit

Attaches to the Auto Scaling group that already exists. Rollback, deployment history and alarm-based reversal, with no application change.

D Nightly window

Makes a slow, irreversible deployment happen when nobody is watching. It hides the problem rather than fixing it.

THE SIGNAL

The word FIRST matters again. A retrofit delivers all three fixes this month; a rewrite delivers them next year.

A security team maintains a hardened base AMI by launching an instance from last quarter's image, applying updates and a hardening script by hand, and creating a new AMI. It takes two days, happens quarterly, and the resulting images differ in ways nobody can explain. Compliance requires evidence that each image meets a STIG baseline.

What should the solutions architect recommend?

- A An EC2 Image Builder pipeline with managed STIG hardening components and tests gating distribution
- B Document the manual steps as a runbook so they are followed consistently
- C Use AWS Marketplace images and skip the hardening step
- D Patch running instances with Patch Manager instead of rebuilding images

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A** Image Builder with STIG components
Repeatable recipe, managed hardening components, and distribution gated on tests, which is the evidence compliance asked for.
- B** Document the manual steps
Makes the ritual more consistent while keeping two days of human work and no test gate. Humans still deviate.
- C** Marketplace, no hardening
Abandons the compliance requirement. A pre-hardened Marketplace image can be the source, and not a replacement for the baseline.
- D** Patch running instances
Answers a different question. New instances would still launch from an unhardened, out-of-date image.

THE SIGNAL

Images that differ in unexplainable ways is the signature of a manual process. A recipe produces the same image every time.

The platform team now produces a golden AMI monthly. An audit finds that only 40% of running instances were launched from it. Teams are using whichever AMI they had last time, including images two years old.

The company runs AWS Organizations with SCPs already in place.

What should the solutions architect recommend?

- A Email the AMI ID to all teams monthly
- B A Config rule detecting unapproved AMIs, plus an SCP denying RunInstances unless the AMI carries the approved tag
- C Delete all non-golden AMIs from every account
- D Rely on Patch Manager to bring the old instances up to date

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Email the AMI ID

Depends on every team reading and acting on an email every month. The audit shows that 60% already do not.

C Delete other AMIs

Breaks running Auto Scaling groups whose launch templates reference them, turning a compliance gap into an outage.

B Config rule plus SCP

The SCP prevents launching from an unapproved image, and the Config rule reports anything already running. Detect and prevent together.

D Patch Manager

Patches the operating system and does nothing about hardening, agents or configuration that the golden image carries.

THE SIGNAL

Producing a standard and enforcing it are different projects. The scenario mentions existing SCPs so you reach for the preventive half.

A critical vulnerability is announced in a library present in the company's base image. Today, somebody notices the advisory, opens a ticket, and the platform team rebuilds the image within about a week. Instances are then replaced over the following fortnight. Security wants this to happen without a human noticing the advisory.

What should the solutions architect implement?

- ☐ A An EventBridge rule on Amazon Inspector High or Critical findings, triggering the Image Builder pipeline, with Auto Scaling instance refresh to roll out
- ☐ B Schedule the Image Builder pipeline to run nightly
- ☐ C Subscribe the platform team to vendor security mailing lists
- ☐ D Run Patch Manager hourly across the fleet

◆ PAUSE AND DECIDE

What should the solutions architect implement?

- A

 Inspector to EventBridge to pipeline
Inspector finds the CVE, EventBridge triggers a rebuild, tests gate distribution, and instance refresh rolls it out. No human reads anything.
- B

 Nightly rebuild
The near miss. It removes the human, and rebuilds every night whether or not anything changed, and still waits up to 24 hours.
- C

 Mailing lists
Exactly the current process. Somebody must read, recognise and act, which is what security asked to remove.
- D

 Hourly Patch Manager
Patches running instances and leaves the base image vulnerable, so every new instance launches with the CVE.

THE SIGNAL

Without a human noticing is the requirement. Only an event-driven trigger removes the human; a schedule merely replaces them with a clock.

A retailer deploys weekly. A new recommendation engine is ready but the business wants it enabled for 5% of customers first, and switched off within seconds if conversion drops. The current process would require a full deployment to enable it and another to disable it, each taking 30 minutes.

The code is already written and tested.

What should the solutions architect recommend?

- A Deploy the code with the feature behind an AppConfig feature flag, rolled out gradually with automatic rollback on a CloudWatch alarm
- B Use a canary deployment to release the new version to 5% of instances
- C Deploy to a separate environment and split traffic with Route 53 weighted records
- D Speed up the pipeline so deployments take 10 minutes

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A** Feature flag with AppConfig
Deploy and release become separate events. Enabling and disabling are configuration changes measured in seconds, not deployments.
- B** Canary deployment
The strong near miss. It reaches 5% of instances rather than 5% of customers, and reverting is still a deployment.
- C** Second environment plus DNS
Duplicates the environment, and DNS caching means you cannot guarantee when a change takes effect. Module eight.
- D** Faster pipeline
Ten minutes is still not seconds, and it accepts that release must equal deployment rather than questioning it.

THE SIGNAL

Within seconds is the constraint. Any answer where turning the feature off is a deployment fails it, however fast the deployment is.

Two years of monthly image builds across four Regions and 30 accounts have produced roughly 2,900 AMIs and 11,000 EBS snapshots. Storage costs are climbing and engineers cannot tell which images are current. Deleting them by hand has been attempted twice and abandoned.

Some older images must be retained for a documented rollback window.

What should the solutions architect recommend?

- ☐ A EC2 Image Builder lifecycle policies that deprecate and delete images beyond the retention window
- ☐ B A Lambda function scanning all accounts and deleting AMIs older than 90 days
- ☐ C Stop building images monthly and build quarterly instead
- ☐ D Move the snapshots to S3 Glacier Deep Archive

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A** Image lifecycle policies
Built for exactly this. Deprecate first, delete after the retention window, across Regions and accounts, with no code to maintain.
- B** Custom Lambda cleanup
Rebuilds the feature, and a blunt age rule risks deleting an image still referenced by a launch template or needed for rollback.
- C** Build quarterly
Slows accumulation and leaves 2,900 existing images in place, while making the base image three months stale.
- D** Glacier for snapshots
EBS snapshots have their own archive tier and are not moved to S3 storage classes. It also does not answer which image is current.

THE SIGNAL

Deprecate then delete is the useful distinction. Deprecation hides an image from selection without removing it during the rollback window.

What to carry out of this module

- ◆ **Name what the process cannot do first.** Retrofit the gap; do not propose a rewrite unless asked.
- ◆ **Image Builder gates distribution on tests.** A failed image never reaches anybody, which is what makes it evidence.
- ◆ **Producing a golden image is not enforcing it.** Config rule to detect, SCP to prevent.
- ◆ **Separate deployment from release.** A feature flag turns rollback from a pipeline run into a configuration change.

ON THE EXAM

Inspector finding to EventBridge to Image Builder to instance refresh is the fully closed loop, built entirely from earlier modules.